

微服务电商平台架构设计文档微服务电商平台架构设计文档

项目代号: ShopStream 版本: v2.0 文档类型: 技术架构设计 设计阶段: 详细设计
最后更新: 2026-06-28**项目代号**: ShopStream
版本: v2.0
文档类型: 技术架构设计
设计阶段: 详细设计
最后更新: 2026-06-28
项目代号: ShopStream 版本: v2.0 文档类型: 技术架构设计 设计阶段: 详细设计
最后更新: 2026-06-28**项目代号**: ShopStream
版本: v2.0
文档类型: 技术架构设计
设计阶段: 详细设计
最后更新: 2026-06-28

1. 项目背景与目标1. 项目背景与目标

1.1 业务背景1.1 业务背景

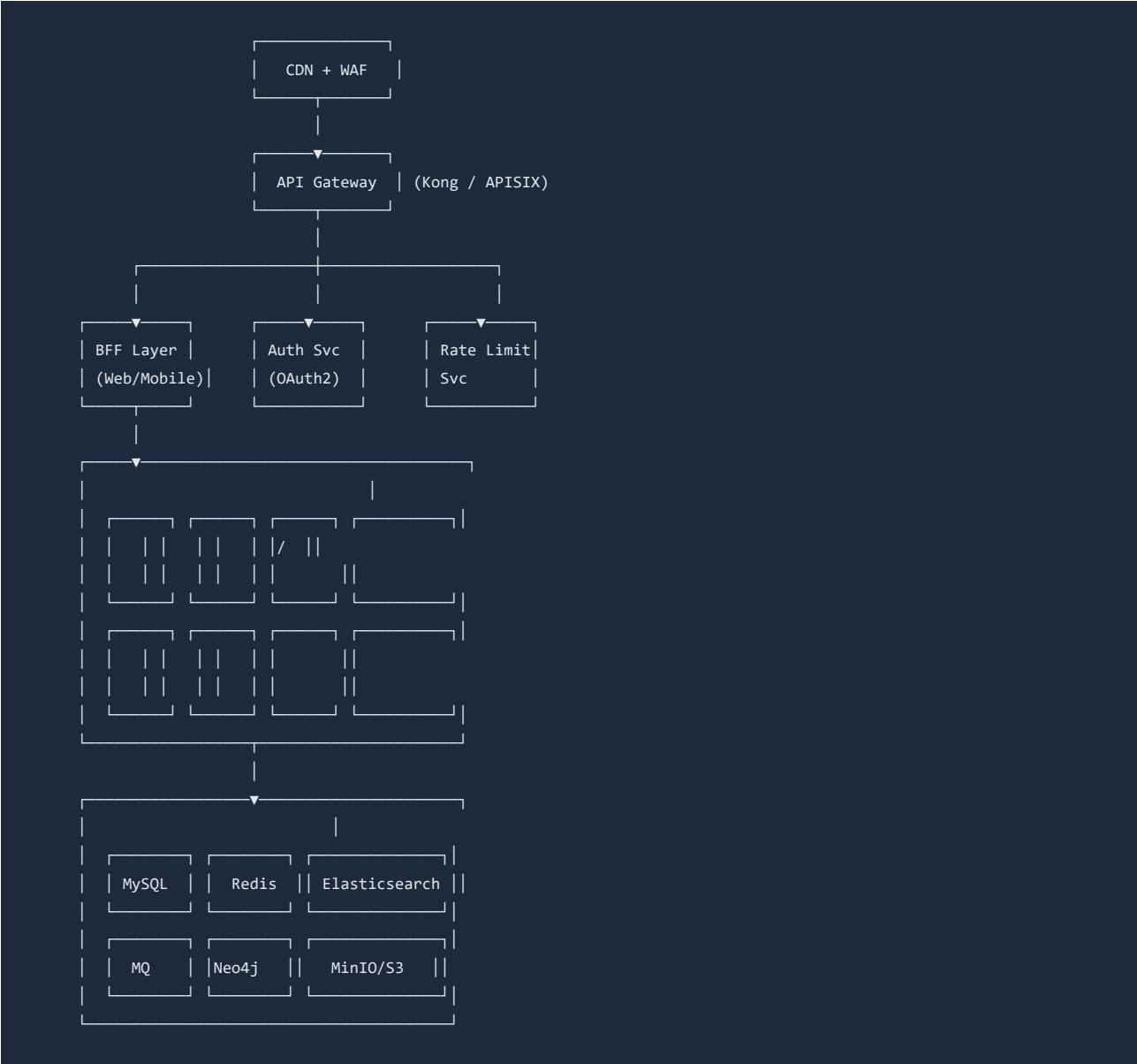
ShopStream 是一个面向中型电商企业的 SaaS 平台, 预计支撑 日均 500 万独立访客、峰值 QPS 20,000 的业务规模。平台需要支持多租户模式, 每租户数据物理隔离。

1.2 架构目标1.2 架构目标

目标目标	指标指标	优先级优先级
高可用高可用	99.95% (年度)99.95% (年度)	P0P0
可扩展可扩展	水平扩展至 100+ 服务节点水平扩展至 10 P0P0	
低延迟低延迟	P99 < 200ms (读) , < 500ms (写) P99 P1P1	
数据一致性数据一致性	订单/支付强一致, 其他最终一致订单/支付 P0P0	
可观测可观测	全链路追踪 + 集中日志 + 指标监控全链路 P1P1	
安全合规安全合规	等保三级 + PCI-DSS等保三级 + PCI-DSS P1P1	

2. 系统架构总览2. 系统架构总览

2.1 架构图2.1 架构图



2.2 技术选型

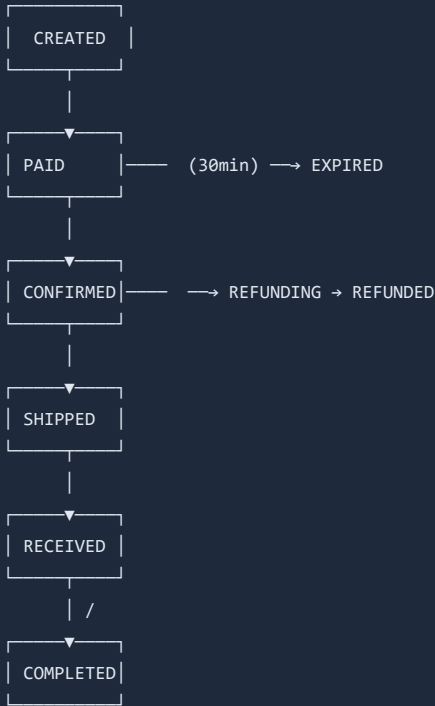
层次	技术	选择理由
语言	Go (核心服务) + TypeScript (BFF)	Go 高性能 + TS 全栈开发效率
服务框架	Go-Zero (Go) / NestJS (BFF)	丰富的微服务治理能力
RPC	gRPC (服务间) + REST (对外)	高性能 + 生态兼容
消息队列	Apache Pulsar	多租户隔离 + 延迟消息
容器编排	Kubernetes + Helm	标准化部署
服务网格	Istio (Ambient)	零侵入流量治理

3. 核心领域设计

3.1 订单服务3.1 订单服务

订单服务是整个平台的核心，需要保证强一致性和幂等性。

3.1.1 订单状态机3.1.1 订单状态机



3.1.2 幂等性设计3.1.2 幂等性设计

```
//
func (s *OrderService) CreateOrder(ctx context.Context, req *CreateOrderReq) (*Order, error) {
    // 1. Key
    idempotentKey := fmt.Sprintf("order:create:%s", req.IdempotentKey)

    // 2. Redis SET NX + TTL
    ok, err := s.redis.SetNX(ctx, idempotentKey, "processing", 5*time.Minute).Result()
    if err != nil {
        return nil, fmt.Errorf("redis error: %w", err)
    }
    if !ok {
        //
        existing, _ := s.repo.GetByIdempotentKey(ctx, req.IdempotentKey)
        if existing != nil {
            return existing, nil
        }
        return nil, ErrOrderProcessing
    }

    // 3.
    order, err := s.repo.CreateInTx(ctx, func(tx *sql.Tx) (*Order, error) {
        //
        if err := s.inventoryClient.Deduct(ctx, req.Items); err != nil {
            return nil, fmt.Errorf("inventory deduct failed: %w", err)
        }
        //
        return tx.InsertOrder(ctx, req)
    })

    // 4. Key
    s.redis.Set(ctx, idempotentKey, order.ID, 24*time.Hour)

    return order, err
}
```

3.2 库存服务3.2 库存服务

并发扣减策略并发扣减策略

策略策略	优点优点	缺点缺点	适用场景适用场景
数据库行锁数据库行锁	最强一致性最强一致性	性能瓶颈性能瓶颈	低并发低并发
Redis + LuaRedis + Lua	高性能高性能	缓存与 DB 可能不一致缓存与 C	高并发高并发
分段锁分段锁	高并发 + 强一致高并发 + 强一致	实现复杂实现复杂	超高并发超高并发
Redis + MQ 异步**Redis + M	性能 + 最终一致**性能 + 最终一致	短暂超卖风险短暂超卖风险	推荐方案推荐方案

4. 数据架构4. 数据架构

4.1 数据库分库策略4.1 数据库分库策略

```
:
:
:
: tenant_{tenant_id}_db

:
: user_id
: user_id % 16
: orders_00 ~ orders_15

:
:
: / / / SKU
```

4.2 缓存策略4.2 缓存策略

数据类型数据类型	缓存时间缓存时间	更新策略更新策略	存储存储
商品基本信息商品基本信息	1 小时1 小时	Cache-Aside + 延迟双删Cache-Aside + 延迟双删	Redis StringRedis String
商品库存商品库存	实时实时	预扣减 + 定时同步预扣减 + 定时同步	Redis HashRedis Hash
用户 Session用户 Session	7 天7 天	JWT 无状态JWT 无状态	Redis StringRedis String
热搜词热搜词	5 分钟5 分钟	定时刷新定时刷新	Redis Sorted SetRedis Sorted Set
页面片段页面片段	24 小时24 小时	CDN 边缘缓存CDN 边缘缓存	CDNCDN

5. 消息与事件5. 消息与事件

5.1 事件驱动架构5.1 事件驱动架构

```
(Apache Pulsar):

order.created →
    →
    → /

order.paid →
    →
    →

order.shipped →
    →
```

5.2 死信队列策略5.2 死信队列策略

```
//
type RetryPolicy struct {
    MaxRetries    int           // : 3
    InitialDelay  time.Duration // : 1s
    MaxDelay      time.Duration // : 60s
    BackoffFactor float64       // : 2.0
    DLQ           string        // : order-events-dlq
}
```

6. 可观测性6. 可观测性

6.1 监控指标体系6.1 监控指标体系

层级层级	指标指标	采集方式采集方式	告警规则告警规则
基础设施基础设施	CPU/Mem/Disk/NetworkCPU	Prometheus Node ExporterPi	> 80%> 80%
应用服务应用服务	QPS/Latency/Error RateQPS/	OpenTelemetry + JaegerOpe	Error > 1%Error > 1%
业务指标业务指标	下单量/支付转化率/GMV下单量	埋点 SDK → ClickHouse埋点	环比下降 > 20%环比下降 > 20%
数据库数据库	连接数/慢查询/死锁连接数/慢查	MySQL ExporterMySQL Expo	慢查询 > 200ms慢查询 > 200ms

6.2 日志规范6.2 日志规范

```
{
  "timestamp": "2026-07-13T09:30:00.123Z",
  "level": "INFO",
  "service": "order-service",
  "trace_id": "abc123def456",
  "span_id": "789ghi",
  "user_id": "u_12345",
  "tenant_id": "t_67890",
  "action": "order.created",
  "duration_ms": 45,
  "status": "success",
  "detail": {
    "order_id": "ord_001",
    "amount": 29900,
    "items_count": 3
  }
}
```

7. 安全设计7. 安全设计

7.1 服务间认证7.1 服务间认证

```
# Istio mTLS + AuthorizationPolicy
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: order-service-policy
  namespace: production
spec:
  selector:
    matchLabels:
      app: order-service
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/production/sa/bff-service"
              - "cluster.local/ns/production/sa/payment-service"
      to:
        - operation:
            methods: ["POST", "GET"]
            paths: ["/api/v1/orders/*"]
```

8. 部署架构8. 部署架构

8.1 环境规划8.1 环境规划

环境环境	用途用途	节点数节点数	规格规格
devdev	开发联调开发联调	33	4C16G4C16G
stagingstaging	预发布验证预发布验证	66	8C32G8C32G
productionproduction	生产环境生产环境	20+20+	16C64G16C64G
drdr	容灾容灾	1010	8C32G8C32G

本文档属于公司机密，未经授权不得外传。架构评审通过日期：2026-06-15